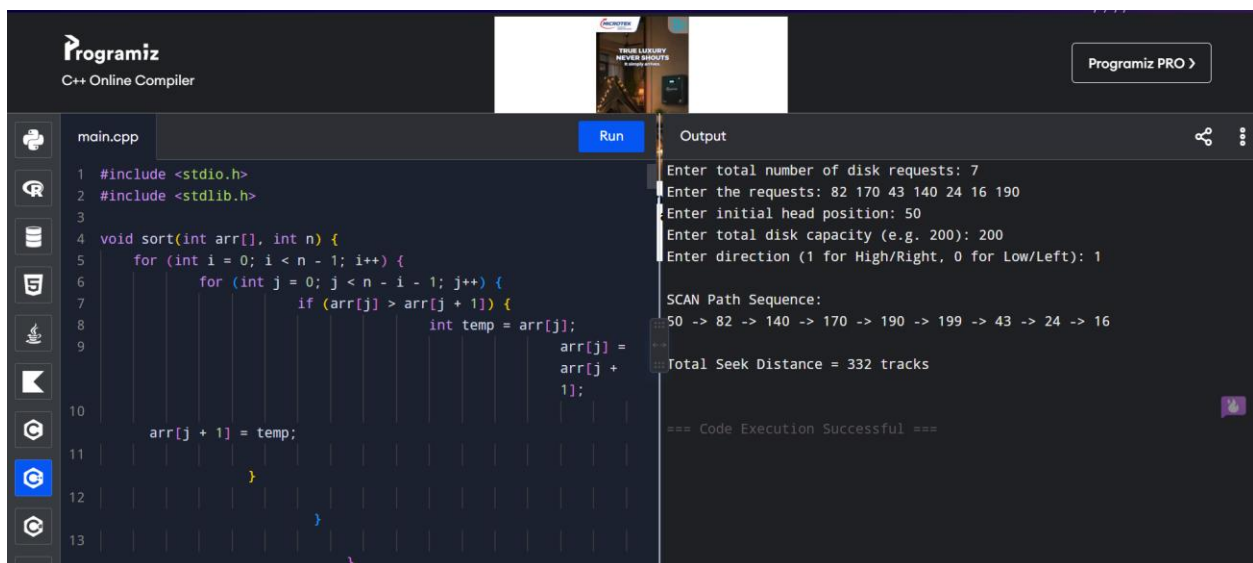


EXPERIMENT 32 :

32. Design a C program to simulate SCAN disk scheduling algorithm.

AIM:

To design a C program to simulate the SCAN disk scheduling algorithm.



```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void sort(int arr[], int n) {
5     for (int i = 0; i < n - 1; i++) {
6         for (int j = 0; j < n - i - 1; j++) {
7             if (arr[j] > arr[j + 1]) {
8                 int temp = arr[j];
9                 arr[j] = arr[j + 1];
10                arr[j + 1] = temp;
11            }
12        }
13    }
```

Output

Enter total number of disk requests: 7
Enter the requests: 82 170 43 140 24 16 190
Enter initial head position: 50
Enter total disk capacity (e.g. 200): 200
Enter direction (1 for High/Right, 0 for Low/Left): 1

SCAN Path Sequence:
50 -> 82 -> 140 -> 170 -> 190 -> 199 -> 43 -> 24 -> 16

Total Seek Distance = 332 tracks

=== Code Execution Successful ===

PSEUDO CODE :

BEGIN

PRINT "Enter total number of disk requests:"

READ num_requests

DECLARE requests array of size num_requests

PRINT "Enter request queue:"

FOR i FROM 0 TO num_requests - 1 DO

 READ requests[i]

END FOR

PRINT "Enter initial head position:"

READ head

PRINT "Enter total disk size (e.g., 200 for 0-199):"

READ disk_size

PRINT "Enter direction (1 for High/Right, 0 for Low/Left):"

READ direction

SORT requests array in ascending order

SET total_movement = 0

SET current_head = head

// Find split index

SET index = 0

FOR i FROM 0 TO num_requests - 1 DO

 IF requests[i] > head THEN

 SET index = i

 BREAK

 END IF

END FOR

PRINT "Head Traversal Path:"

PRINT current_head

IF direction == 1 THEN // High/Right Direction

 FOR i FROM index TO num_requests - 1 DO

 SET total_movement = total_movement + ABSOLUTE(requests[i] -
current_head)

 SET current_head = requests[i]

 PRINT " -> " + current_head

 END FOR

// Go to end of disk

```
    SET total_movement = total_movement + ABSOLUTE((disk_size - 1) -  
current_head)
```

```
    SET current_head = disk_size - 1
```

```
    PRINT " -> " + current_head
```

```
// Reverse and process left side
```

```
FOR i FROM index - 1 DOWNTO 0 DO
```

```
    SET total_movement = total_movement + ABSOLUTE(requests[i] -  
current_head)
```

```
    SET current_head = requests[i]
```

```
    PRINT " -> " + current_head
```

```
END FOR
```

```
ELSE // Low/Left Direction
```

```
FOR i FROM index - 1 DOWNTO 0 DO
```

```
    SET total_movement = total_movement + ABSOLUTE(requests[i] -  
current_head)
```

```
    SET current_head = requests[i]
```

```
    PRINT " -> " + current_head
```

```
END FOR
```

```
// Go to start of disk
```

```
SET total_movement = total_movement + ABSOLUTE(0 - current_head)
```

```
SET current_head = 0
```

```
PRINT " -> 0"
```

```
// Reverse and process right side
```

```
FOR i FROM index TO num_requests - 1 DO
```

```
    SET total_movement = total_movement + ABSOLUTE(requests[i] -  
current_head)
```

```
    SET current_head = requests[i]
```

```
        PRINT " -> " + current_head
    END FOR
END IF
```

```
    PRINT "Total Head Movement: " + total_movement
END
```

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void sort(int arr[], int n)
```

```
{
```

```
    for (int i = 0; i < n - 1;
```

```
    i++) {
```

```
        for (int j = 0; j < n -
```

```
        i - 1; j++) {
```

```
            if (arr[j] > arr[j +
```

```
            1]) {
```

```
                int temp =
```

```
                arr[j];
```

```
                arr[j] = arr[j +
```

```
                1];
```

```
        arr[j + 1] =  
temp;  
    }  
}  
}  
}
```

```
int main() {  
  
    int requests[30],  
num_requests, head,  
disk_size, direction;  
  
    int total_movement =  
0;  
  
    printf("Enter total  
number of disk requests:  
");  
  
    scanf("%d",  
&num_requests);
```

```
printf("Enter the  
requests: ");
```

```
for (int i = 0; i <  
num_requests; i++) {
```

```
scanf("%d",  
&requests[i]);
```

```
}
```

```
printf("Enter initial  
head position: ");
```

```
scanf("%d", &head);
```

```
printf("Enter total  
disk capacity (e.g. 200):
```

```
");
```

```
scanf("%d",  
&disk_size);
```

```
    printf("Enter direction  
(1 for High/Right, 0 for  
Low/Left): ");
```

```
    scanf("%d",  
&direction);
```

```
    sort(requests,  
num_requests);
```

```
    int index = 0;
```

```
    for (int i = 0; i <  
num_requests; i++) {
```

```
        if (requests[i] >  
head) {
```

```
            index = i;
```

```
            break;
```

```
        }
```

```
    }
```

```

    int current_head =
head;

    printf("\nSCAN Path
Sequence:\n%d",
current_head);

    if (direction == 1) { //
Right/High

        for (int i = index; i
< num_requests; i++) {

            total_movement
+= abs(requests[i] -
current_head);

            current_head =
requests[i];

            printf(" -> %d",
current_head);

        }

        total_movement +=
abs((disk_size - 1) -
current_head);

```

```
        current_head =  
disk_size - 1;
```

```
        printf(" -> %d",  
current_head);
```

```
        for (int i = index -  
1; i >= 0; i--) {
```

```
            total_movement  
+= abs(requests[i] -  
current_head);
```

```
            current_head =  
requests[i];
```

```
            printf(" -> %d",  
current_head);
```

```
        }
```

```
    } else { // Left/Low
```

```
        for (int i = index -  
1; i >= 0; i--) {
```

```
            total_movement  
+= abs(requests[i] -
```

```
current_head);

    current_head =
requests[i];

    printf(" -> %d",
current_head);

    }

    total_movement +=
abs(0 - current_head);

    current_head = 0;

    printf(" -> 0");

for (int i = index; i
< num_requests; i++) {

    total_movement
+= abs(requests[i] -
current_head);

    current_head =
requests[i];

    printf(" -> %d",
current_head);
```

```
}
```

```
}
```

```
    printf("\n\nTotal Seek  
Distance = %d  
tracks\n",  
total_movement);  
  
    return 0;  
  
}
```